

Week 5: Tree Learning

CSCI 447: Machine Learning– Fall 2026

1

Decision Tree Learning

- Rule Induction
 - Explanation Based Learning
 - Speed up classification
 - Domain theory → Explains examples
 - Sequential Covering
-
- Iterative Dichotomizer 3 (ID3) – J. R. Quinlan
 - Classification and Regression Tree (CART) – L. Breiman, J. H. Friedman, R. A. Olshen, C. J. Stone

2

Classification Tree

- A tree structure where
 - Interior nodes are decision variables (need not be binary)
 - Leaf nodes assign class labels
- Assume
 - Data set $\mathcal{D}$
 - Feature set $\mathcal{F}$, where $f_i \in \mathcal{F} \rightarrow \text{domain } d(f_i)$
- When we choose a feature $f_i \in \mathcal{F}$, we partition $\mathcal{D}$ into $|d(f_i)|$ subsets (again, need not just be two)

Building the Tree - Complexity

- Begin by considering the complexity of learning a decision tree
 - Sample complexity
 - Algorithmic complexity
- How do we build the tree?
- Could choose features at random
 - Not efficient
 - Could lead to overfitting
- Goal: Build a tree that uses the fewest attributes possible

The Decision Tree Problem

- Let $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ be a finite set of training examples
- Let $\mathcal{F} = \{f_1, \dots, f_k\}$ be a finite set of features
- Let w be the cost of some tree T = expected number of features required to classify example $\mathbf{x}_i$

$DT(\mathcal{D}, \mathcal{F}, w)$: Given example set $\mathcal{D}$ and feature set $\mathcal{F}$, does there exist a decision tree using only features from $\mathcal{F}$ for classifying examples in $\mathcal{D}$ such that the cost of the decision tree is less than w ?

Theorem: $DT(\mathcal{D}, \mathcal{F}, w)$ is NP-complete (Hyafil & Rivest, 1976).

Proof Sketch

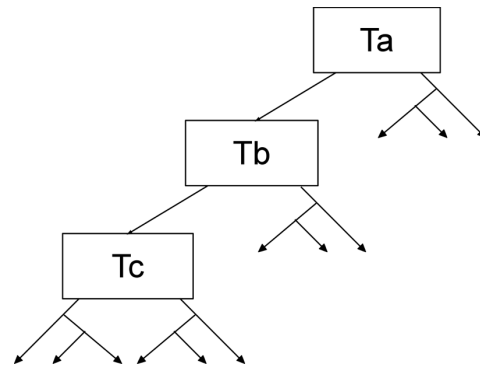
First, we show that $DT(\mathcal{D}, \mathcal{F}, w)$ is in $\mathcal{NP}$

- Compute the length of each branch of the tree
- Multiply each branch/path length by the fraction of training examples at the leaves
- Add up all the weighted branch lengths
- Complexity: Linear in the size of the tree via depth-first search (technically, $O(n|\mathcal{F}|)$ if each leaf has one example, and each branch uses all $|\mathcal{F}|$ features; still polynomial)

Proof Sketch

Second, we need a polynomial-time reduction from another $\mathcal{NP}$ -hard problem

- Use the Exact Cover 3 problem
- Given a set of $3q$ elements and a collection of subsets, finding a grouping of these elements such that each element belongs to exactly one of these subsets and each of these subsets has exactly three elements.



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

7

Example

- Data characterizing the weather
- Features: *Outlook, Temperature, Humidity, Wind*
- Domains:
 - $d(\text{Outlook}) = \{\text{sunny, overcast, rainy}\}$
 - $d(\text{Temperature}) = \{\text{cold, mild, hot}\}$
 - $d(\text{Humidity}) = \{\text{high, normal}\}$
 - $d(\text{Wind}) = \{\text{true, false}\}$
- Assume binary (two-class) classification. Note, this need not be the case.

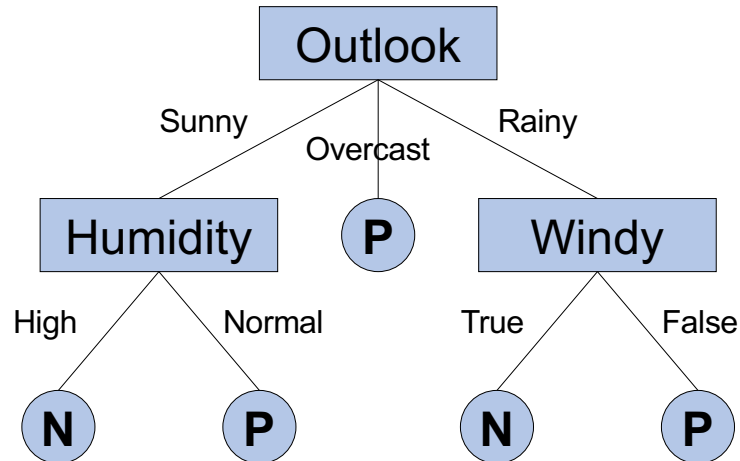


MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

8

Example



9

The ID3 Algorithm

- Employ greedy search
- Start by assuming only two classes (will generalize next)
- Based on Shannon's Information Theory (reduce entropy)
 - Let $I(p, n)$ be the entropy (amount of uncertainty) in the data

$$I(p, n) = -\frac{p}{p+n} \lg \frac{p}{p+n} - \frac{n}{p+n} \lg \frac{n}{p+n}$$

- Consider $f_i \in \mathcal{F}$, generating a partition $\mathcal{D}_\pi \subseteq \mathcal{D}$

$$E_\pi(f_i) = \sum_{j=1}^{m_i} \frac{p_\pi^j + n_\pi^j}{p_\pi + n_\pi} I(p_\pi^j, n_\pi^j)$$

where $m_i = |d(f_i)|$, $p_\pi, n_\pi \sim \mathcal{D}_\pi$.

- Information gain: $gain_\pi(f_i) = I(p_\pi, n_\pi) - E_\pi(f_i)$

10

More than Two Classes

- Need to consider the entropy of a class structure with $k > 2$ classes

$$I(c_1, \dots, c_k) = - \sum_{\ell=1}^k \frac{c_\ell}{c_1 + \dots + c_k} \lg \frac{c_\ell}{c_1 + \dots + c_k}$$

- Also need expected entropy

$$E_\pi(f_i) = \sum_{j=1}^{m_i} \frac{c_{\pi,1}^j + \dots + c_{\pi,k}^j}{c_{\pi,1} + \dots + c_{\pi,k}} I(c_{\pi,1}^j + \dots + c_{\pi,k}^j)$$

11

Example

- Recall:

- $d(\text{Outlook}) = \{\text{sunny, overcast, rainy}\}$
- $d(\text{Temperature}) = \{\text{cold, mild, hot}\}$
- $d(\text{Humidity}) = \{\text{high, normal}\}$
- $d(\text{Wind}) = \{\text{true, false}\}$

- Let **P** denote the positive class
- Let **N** denote the negative class

$$I(p, n) = - \frac{9}{14} \lg \frac{9}{14} - \frac{5}{14} \lg \frac{5}{14} \approx 0.94$$

Example	Outlook	Temperature	Humidity	Wind	Class
1	Sunny	Hot	High	False	N
2	Sunny	Hot	High	True	N
3	Overcast	Hot	High	False	P
4	Rainy	Mild	High	False	P
5	Rainy	Cool	Normal	False	P
6	Rainy	Cool	Normal	True	N
7	Overcast	Cool	Normal	True	P
8	Sunny	Mild	High	False	N
9	Sunny	Cool	Normal	False	P
10	Rainy	Mild	Normal	False	P
11	Sunny	Mild	Normal	True	P
12	Overcast	Mild	High	True	P
13	Overcast	Hot	Normal	False	P
14	Rainy	Mild	High	True	N

12

Example

- Consider the feature *Outlook*

$$E(\text{Outlook}) = \frac{5}{14} I_{\text{sunny}}(p, n) + \frac{4}{14} I_{\text{overcast}}(p, n) + \frac{5}{14} I_{\text{rainy}}(p, n)$$

- Consider the three partitions

$$I_{\text{sunny}}(p, n) = -\frac{2}{5} \lg \frac{2}{5} - \frac{3}{5} \lg \frac{3}{5} \approx 0.971$$

$$I_{\text{overcast}}(p, n) = -\frac{4}{4} \lg \frac{4}{4} - \frac{0}{4} \lg \frac{0}{4} \approx 0.0$$

$$I_{\text{rainy}}(p, n) = -\frac{3}{5} \lg \frac{3}{5} - \frac{2}{5} \lg \frac{2}{5} \approx 0.971$$

- Then

$$E(\text{Outlook}) = \frac{5}{14} 0.971 + \frac{4}{14} 0.0 + \frac{5}{14} 0.971 \approx 0.694$$

- So the gain is $\text{gain}(\text{Outlook}) = 0.94 - 0.694 = 0.246$

Example	Outlook	Temperature	Humidity	Wind	Class
1	Sunny	Hot	High	False	N
2	Sunny	Hot	High	True	N
3	Overcast	Hot	High	False	P
4	Rainy	Mild	High	False	P
5	Rainy	Cool	Normal	False	P
6	Rainy	Cool	Normal	True	N
7	Overcast	Cool	Normal	True	P
8	Sunny	Mild	High	False	N
9	Sunny	Cool	Normal	False	P
10	Rainy	Mild	Normal	False	P
11	Sunny	Mild	Normal	True	P
12	Overcast	Mild	High	True	P
13	Overcast	Hot	Normal	False	P
14	Rainy	Mild	High	True	N



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

13

Example

- Need the expected entropies for all four features

$$E(\text{Outlook}) \approx 0.694$$

$$E(\text{Temperature}) \approx 0.911$$

$$E(\text{Humidity}) \approx 0.789$$

$$E(\text{Wind}) \approx 0.892$$

- From this, we calculate the gain for all four features

$$\text{gain}(\text{Outlook}) = 0.94 - 0.694 = 0.246$$

$$\text{gain}(\text{Temperature}) = 0.94 - 0.911 = 0.029$$

$$\text{gain}(\text{Humidity}) = 0.94 - 0.789 = 0.151$$

$$\text{gain}(\text{Wind}) = 0.94 - 0.892 = 0.048$$

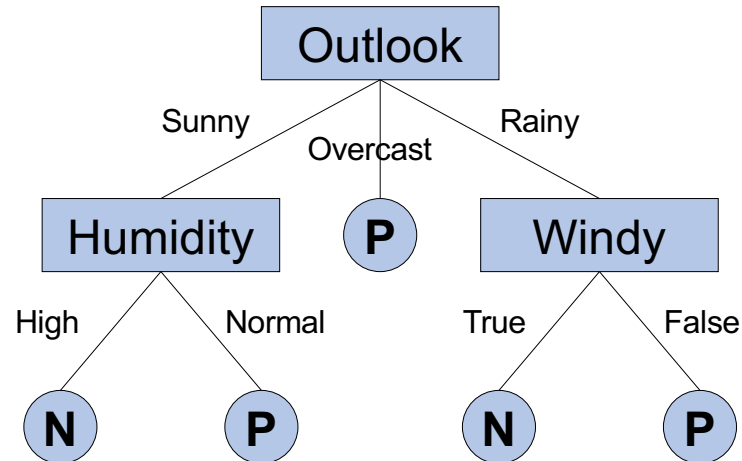


MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

14

Example



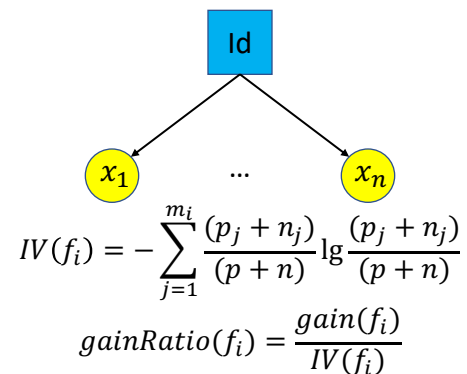
MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

15

Gain Ratio

We need to be careful not to use "features" that correspond to unique identifiers as decision variables in the tree



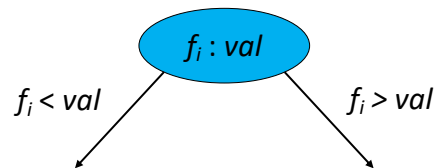
MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

16

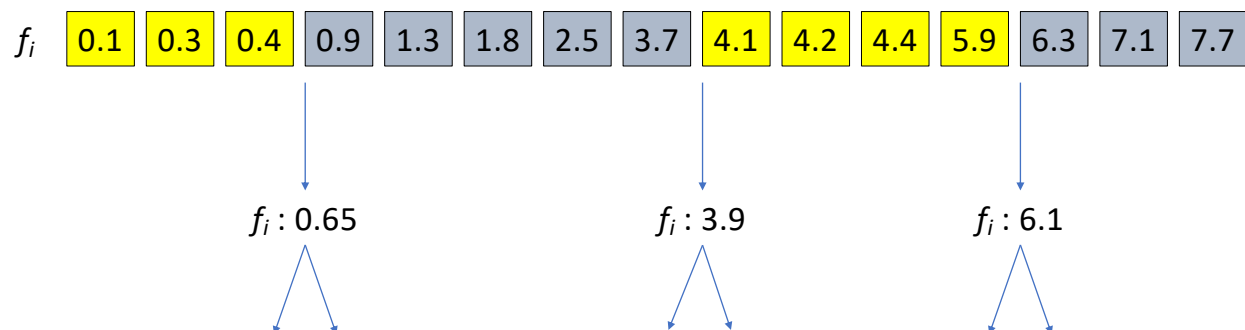
Continuous Attributes

- Standard decision trees assume discrete attributes.
- Decision trees are able to handle continuous attributes as well.
- Generally, when dealing with continuous attributes, binary splits are generated for those attributes.



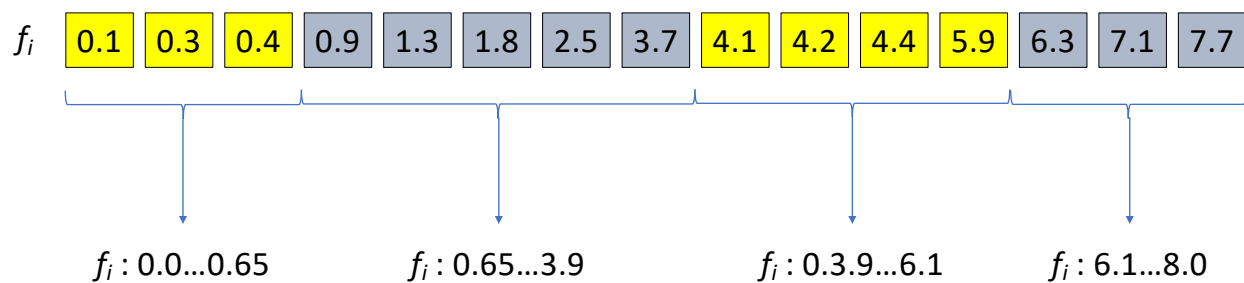
17

Continuous Attributes – Classification



18

Continuous Attributes – Classification



19

Regression Tree Learning

- The task is to learn a tree that predicts real-valued function $f(\mathbf{x})$
- The primary change over classification trees is the splitting criterion
- Linear Regression
 - Ordinary Linear Regression (OLR)
 - Ordinary Least Squares (OLS)
- Training set $\mathcal{D} = \{\mathbf{x}^t, r^t\}_{t=1}^n$, where $r^t \in \mathbb{R}$
- Response function is assumed to have form $r^t = f(\mathbf{x}^t) + \epsilon$
- Functional form is linear: $f(\mathbf{x}^t) = \alpha_0 + \sum_{i=1}^d \alpha_i x_i^t$
- Find suitable values for $\{\alpha_0, \alpha_1, \dots, \alpha_d\}$
- Loss function: $MSE(\alpha) = \frac{1}{n} \sum_{t=1}^n (f(\mathbf{x}^t) - r^t)^2$

20

Maximum Likelihood Estimation

- Recall assumption that $r^t = f(\mathbf{x}^t) + \epsilon$
- Maximum likelihood hypothesis says

$$\begin{aligned}
 h_{ML} &= \arg \max_{h \in \mathcal{H}} P(\mathcal{D}|h) \\
 &= \arg \max_{h \in \mathcal{H}} \prod_{t=1}^n P(\mathbf{x}^t|h) \\
 &= \arg \max_{h \in \mathcal{H}} \prod_{t=1}^n \frac{1}{\sigma\sqrt{2\pi}} \exp \left[-\frac{1}{2} \left(\frac{f(\mathbf{x}^t) - r^t}{\sigma} \right)^2 \right]
 \end{aligned}$$

21

Maximum Likelihood Estimation

Simplifying

$$\begin{aligned}
 h_{ml} &= \arg \max_{h \in \mathcal{H}} \sum_{t=1}^n \ln \frac{1}{\sigma\sqrt{2\pi}} - \frac{1}{2} \left(\frac{f(\mathbf{x}^t) - r^t}{\sigma} \right)^2 \\
 &= \arg \max_{h \in \mathcal{H}} \sum_{t=1}^n -\frac{1}{2} \left(\frac{f(\mathbf{x}^t) - r^t}{\sigma} \right)^2 \\
 &= \arg \max_{h \in \mathcal{H}} \sum_{t=1}^n -(f(\mathbf{x}^t) - r^t)^2 \\
 &= \arg \min_{h \in \mathcal{H}} \sum_{t=1}^n (f(\mathbf{x}^t) - r^t)^2
 \end{aligned}$$

Gradient descent rule: $\nabla_{\alpha} MSE(\alpha) = \frac{2}{n} \sum_{t=1}^n (\alpha^T \mathbf{x}^t - r^t) \mathbf{x}^t$

22

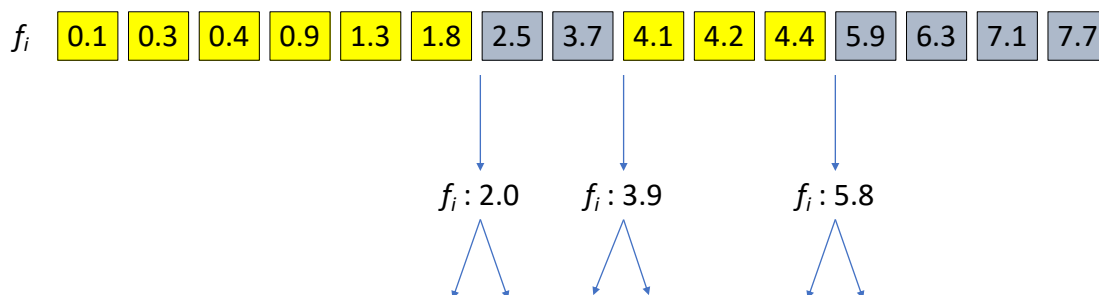
Real-Valued Features

- Binary splits only
 - Sort data by the feature in question
 - Consider successive midpoints
- Can still lead to excessive splits to evaluation
- One strategy is to consider quantiles
- Another strategy is to consider the mean, mean minus one standard deviation, and mean plus one standard deviation

23

Continuous Attributes – Regression

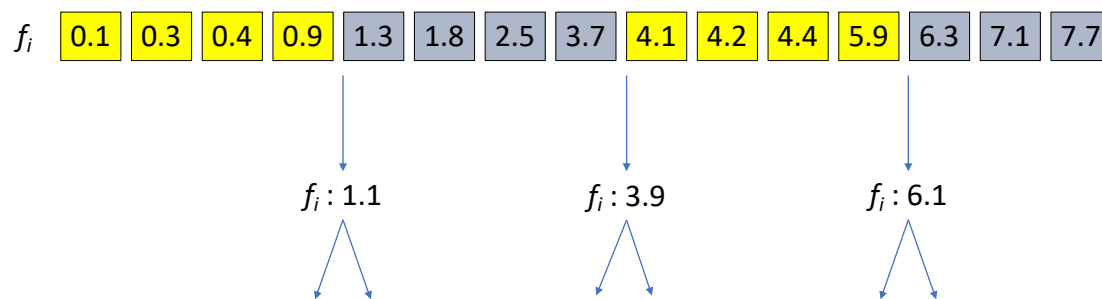
Equal width: four bins, every 1.9 from baseline



24

Continuous Attributes – Regression

Equal frequency: four bins, every 4 points from baseline



25

Using a Regression Tree

- Piecewise constant regressogram
- Local least squares
 - Small disjuncts problem
 - Typically use the “mean” of the points at the leaves

$$\hat{r} = \frac{\sum_t b_m(\mathbf{x}^t) r^t}{\sum_t b_m(\mathbf{x}^t)}$$

where

$$b_m(\mathbf{x}) = \begin{cases} 1 & \text{if } x \in \mathcal{D}_m \\ 0 & \text{otherwise} \end{cases}$$

26

Splitting Criterion

- Use mean squared error (MSE) with “early stopping”
- Let $\mathcal{D}_{mj} \subseteq \mathcal{D}_m$ be a subset of the points at node m in the tree when taking branch j .
- Then

$$\bigcup_{j=1}^p \mathcal{D}_{mj} = \mathcal{D}_m$$

- Let

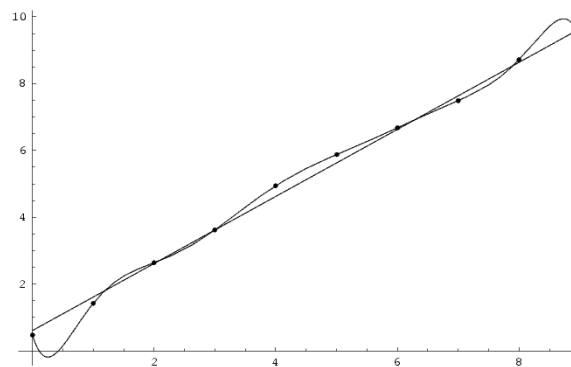
$$b_{mj}(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{x} \in \mathcal{D}_{mj} \\ 0 & \text{otherwise} \end{cases}$$

- Then estimate the response as $\hat{r} = \frac{\sum_t b_m(\mathbf{x}^t) r^t}{\sum_t b_m(\mathbf{x}^t)}$
- Make split that minimizes $MSE'_m = \frac{1}{n_m} \sum_j \sum_t (r^t - \hat{r}_{mj}^t)^2 b_{mj}(\mathbf{x}^t)$

27

Overfitting

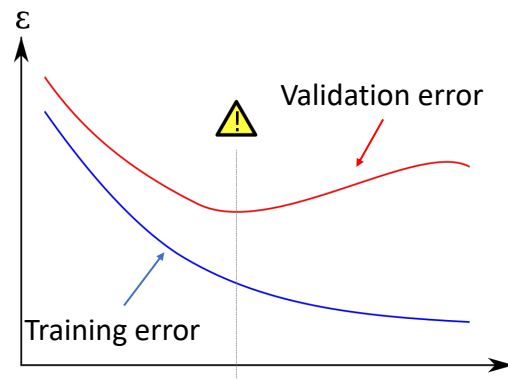
- We say a tree $\mathcal{T}$ overfits to data $\mathcal{D}$ if $\exists \mathcal{T}'$ such that
 - $error_{\mathcal{D}}(\mathcal{T}) < error_{\mathcal{D}}(\mathcal{T}')$ but
 - $error_{\mathcal{X}}(\mathcal{T}) > error_{\mathcal{X}}(\mathcal{T}')$
- In other words, while error on the training data is low, generalization error is high
- Results in fitting the “noise” in the data
- Often tied to the number of parameters in the model



28

Using a Validation Set

- To avoid overfitting, one approach is to maintain a separate “validation” set
- Track training error, $error_D(\mathcal{T})$ relative to the validation error, $error_X(\mathcal{T})$
- When curves diverge, overfitting is starting to occur
- Issues:
 - If data is limited, validation set makes matters worse
 - Comparing algorithms where one uses a validation set and the other does not is unfair
- Alternative: Regularization



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

29

Reduced Error Pruning

- Also known as “post-pruning”
- Grow tree until it cannot be grown any further (overfits)
- Tag vertices for possible pruning
- Test possible pruned trees by removing tagged vertices one at a time
- Test the best against the original on validation set
- Accept pruned tree if no worse
- Repeat until performance on validation set starts to degrade



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

30

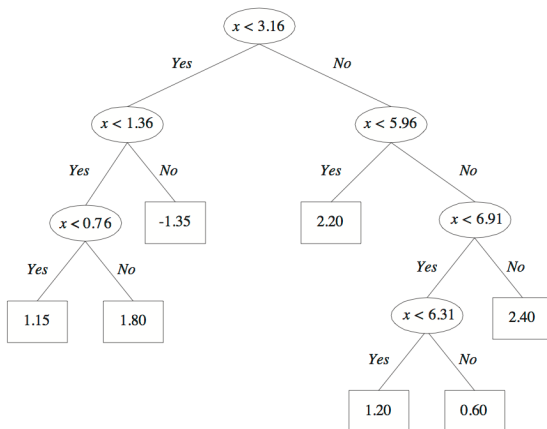
Early Stopping – Regression

- Also known as “pre-pruning”
- Approach commonly used on regression trees, but can be done with classification trees too
- Can be done without a validation set (but can be done with a validation set too)
- For regression, loss function is $MSE(\alpha)$
- Set a threshold (tuned) θ_r to be used as a stopping criterion
- Once $MSE(\alpha)$ drops below θ_r , stop

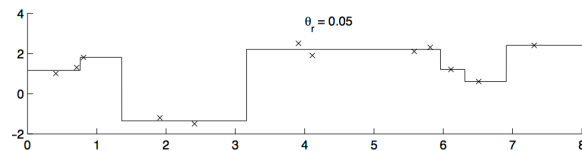
31

Example

Let $\theta_r = 0.05$



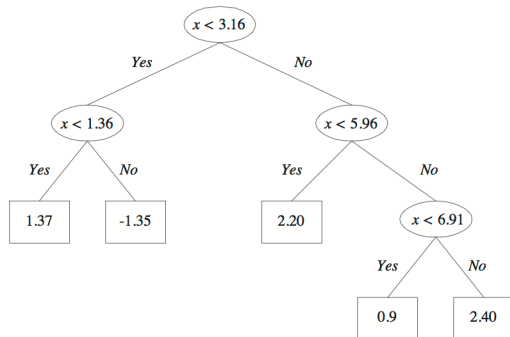
Regressogram



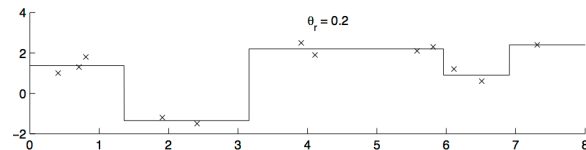
32

Example

Let $\theta_r = 0.2$



Regressogram



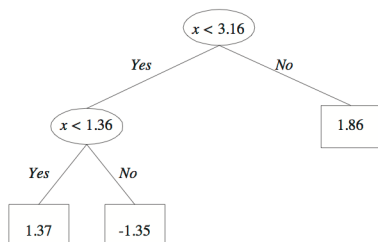
MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

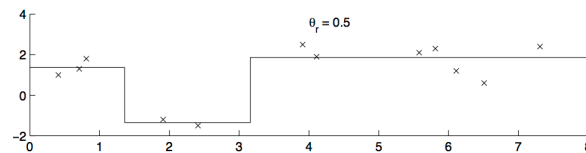
33

Example

Let $\theta_r = 0.5$



Regressogram



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

34

Early Stopping – Classification

- Quinlan proposed an early stopping rule for ID3
- Proposed using a χ^2 test
- H_0 : A is independent of class of objects $\mathcal{C}$ being split by A
- Suppose we test at a 99% confidence level

$$p'_i = p \cdot \frac{p_i + n_i}{p + n}$$

$$n'_i = n \cdot \frac{p_i + n_i}{p + n}$$

$$\chi^2 = \sum_{i=1}^v \left[\frac{(p_i - p'_i)^2}{p'_i} + \frac{(n_i - n'_i)^2}{n'_i} \right]$$

- Assumes $v - 1$ degrees of freedom, where v is the number of values A can take on

35

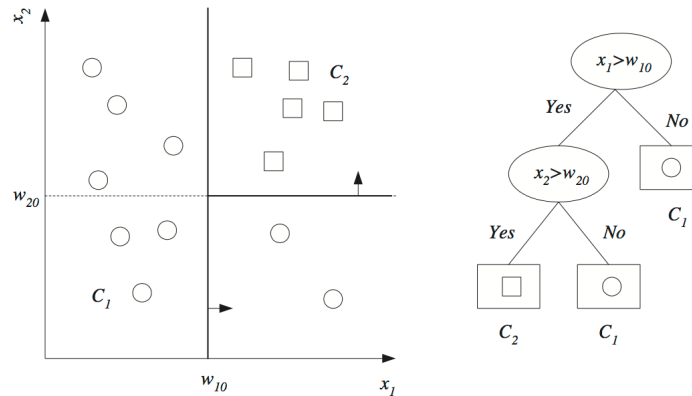
Preference Bias

- How we select the attributes in the tree
- Using information gain or MSE is a “greedy” choice
- Information gain incorporates the preference bias to reduce uncertainty/entropy in the data
- MSE incorporates the preference bias to reduce error
- Both tend to “prefer” shallow trees → supports generalization?

36

Representation Bias

Most decision trees assume univariate splits (also called “axis parallel”)



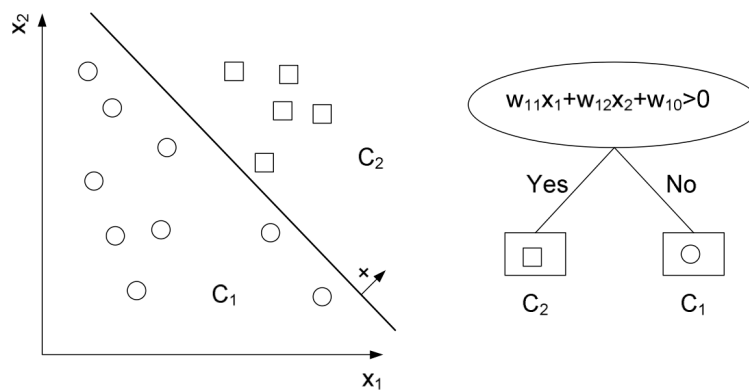
MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

37

Oblique (Multi-variate) Trees

Replace each node with a split of the form $\alpha_0 + \sum_{i=1}^d \alpha_i x_i < \theta$



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

38

Oblique Classifier 1 (OC1)

To find a split of a set of examples T :

- Find the best axis-parallel split of T . Let I be the impurity of this split.
- Repeat R times:
 - Choose a random hyperplane H .
 - (For the first iteration, initialize H to be the best axis-parallel split.)
 - Step 1: Until the impurity measure does not improve, do:
 - Perturb each of the coefficients of H in sequence.
 - Step 2: Repeat at most J times:
 - Choose a random direction and attempt to perturb H in that direction.
 - If this reduces the impurity of H , go to Step 1.
 - Let I_1 = the impurity of H . If $I_1 < I$, then set $I = I_1$.
- Output the split corresponding to I .

Perturbation

Perturb(H, m)

- For $j = 1, \dots, n$
 - Compute U_j (Eq. 3.2)
 - Sort $U_1, \dots, U_n$ in non-decreasing order.
 - a'_m = best univariate split of the sorted U_j s.
 - H_1 = result of substituting a'_m for a_m in H .
 - If ($\text{impurity}(H_1) < \text{impurity}(H)$)
 - { $a_m = a'_m$; $P_{\text{move}} = P_{\text{stag}}$ }
 - Else if ($\text{impurity}(H) = \text{impurity}(H_1)$)
 - { $a_m = a'_m$ with probability P_{move}
 - $P_{\text{move}} = P_{\text{move}} - 0.1 * P_{\text{stag}}$ }

$$V_j = \alpha_0 + \sum_{i=1}^d \alpha_i x_i \quad a_m > \frac{a_m x_{jm} - V_j}{x_{jm}}$$

Logic Motivation

- One of the original problem areas in AI was mathematical theorem proving: Logic Theorist, GPS.
- First complete inference procedure was computational.
- Early on many researchers realized that it was essential to have a “formal” language for talking about knowledge.
- Logic seems like the obvious language.
- Logic and logical inference are generally viewed as essential to symbolic AI.
- Logic is also something that, compared to models like neural networks, tends to be “easy” to understand, thus supporting *explainable* methods in machine learning.

Propositional Logic

- The simplest formal logic.
- Largely based on set theory.
- Forms the foundation for most other logics.
- Will present syntax and semantics.

Syntax of Propositional Logic

- The constants T and F are sentences.
- A propositional symbol (e.g., P , Q , R) is a sentence.
- If α is a sentence and β is a sentence, then the following are sentences.
 - (α)
 - $\neg\alpha$
 - $\alpha \vee \beta$

Propositional Logic Semantics

- Logical constants T and F have fixed interpretations (i.e., True and False respectively).
- If sentence α is True, then so is (α) .
- If sentence α is False, then $\neg\alpha$ is True.
- If either α or β is True, then $\alpha \vee \beta$ is True.

Truth Tables

Negation

α	$\neg \alpha$
T	F
F	T

Disjunction

α	β	$\alpha \vee \beta$
T	T	T
T	F	T
F	T	T
F	F	F



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

45

More Truth Tables

Conjunction

α	β	$\alpha \wedge \beta$
T	T	T
T	F	F
F	T	F
F	F	F

$$\neg(\neg\alpha \vee \neg\beta)$$

Implication

α	β	$\alpha \Rightarrow \beta$
T	T	T
T	F	F
F	T	T
F	F	T

$$\neg\alpha \vee \beta$$

Equivalence

α	β	$\alpha \Leftrightarrow \beta$
T	T	T
T	F	F
F	T	F
F	F	T

$$\neg(\neg(\neg\alpha \vee \beta) \vee \neg(\neg\beta \vee \alpha))$$



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

46

Propositional Inference

- Double-Negation: $[\neg\neg\alpha] \vdash \alpha$
- AND-Elim: $[\alpha_1 \wedge \dots \wedge \alpha_n] \vdash \alpha_i$
- AND-Intro: $[\alpha_1, \dots, \alpha_n] \vdash [\alpha_1 \wedge \dots \wedge \alpha_n]$
- OR-Intro: $[\alpha_i] \vdash [\alpha_1 \vee \dots \vee \alpha_i \vee \dots \vee \alpha_n]$
- Modus Ponens: $[\alpha \Rightarrow \beta, \alpha] \vdash \beta$
- Modus Tollens: $[\alpha \Rightarrow \beta, \neg\beta] \vdash \neg\alpha$
- Unit Resolution: $[\alpha \vee \beta, \neg\beta] \vdash \alpha$
- Resolution: $[\alpha \vee \beta, \neg\beta \vee \delta] \vdash \alpha \vee \delta$
- Subsumption: $[(\alpha \wedge \beta) \vee \beta] \vdash \beta$

Normal Forms

- Conjunctive Normal Form (CNF)
 - A conjunction of disjunctions (clauses) of literals.
 - $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$
- Disjunctive Normal Form (DNF)
 - A disjunction of conjunctions (terms) of literals.
 - $(A \wedge B) \vee (A \wedge \neg C) \vee (A \wedge \neg D) \vee (\neg B \wedge \neg C) \vee (\neg B \wedge \neg D)$
- Horn Form
 - A conjunction of Horn clauses (clauses with ≤ 1 positive literal)
 - $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$
 - Often written as a set of implications (i.e., implicative normal form)
 - $B \Rightarrow A$ and $(C \wedge D) \Rightarrow B$

First-Order Logic

- Also called Predicate Calculus
- Extend language beyond propositions
 - Constants (individuals in the world)—*KingJohn*, 2, *MSU*, ...
 - Variables (e.g., x , y , *row*, *col*)
 - Functions (map individuals to individuals)—*Sqrt*, *LeftLegOf*, ...
 - Predicates (map individuals to truth values)—*Brother*, $>$, $=$, ...
 - Connectives ($\wedge$, $\vee$, $\neg$, $\Rightarrow$, $\Leftrightarrow$)
 - Quantifiers (universal " $\forall$ " and existential " $\exists$ ")
 - Equality (i.e., $=$)

Quantifiers

- Universals are like conjunction
 - $\forall x P(x)$ means P holds for all values x
 - Usually used with implication
- Existentials are like disjunction
 - $\exists x Q(x)$ means Q holds for some value x
 - Usually used with conjunction
- Switching the order of **like** quantifiers does not change meaning.
- Switching the order of **different** quantifiers does.

Inference Rules in FOL

- PL inference rules apply in FOL as well.
- Universal Elim: If $\forall x P(x)$ is true, then $P(x)$ is true for any constant c in the domain. (Like AND Elimination)
- Existential Intro: If $P(c)$ is true, then $\exists x P(x)$ is true. (Like OR Introduction)
- Existential Elim: If $\exists x P(x)$ is true, then $P(c)$ is true for some constant c not appearing in any other sentence (Skolem constant).

Inference Rules in FOL

- Generalized Modus Ponens
 - Combines AND-Intro, Universal-Elim, and Modus Ponens
 - Ex. From $P(c)$, $Q(c)$, and $\forall x (P(x) \wedge Q(x)) \Rightarrow R(x)$ derive $R(c)$.
 - Let $subst(\theta, \alpha)$ denote substitutions resulting from applying some substitution list θ to sentence α .
 - Given atomic sentences $P_1, \dots, P_n$ and implication $(Q_1 \wedge \dots \wedge Q_n) \rightarrow R$ and $subst(\theta, P_i) = subst(\theta, Q_i)$, derive $subst(\theta, R)$.
- GMP requires the ability to find substitutions allowing “facts” to match left-hand-sides of rules.
- This matching process is called “unification.”

Resolution Theorem Proving

- Proof by contradiction
- Unification used to match terms in clauses
- Procedure:
 - Negate the theorem to be proven
 - Add to axiom set
 - Convert all axioms to clause form
 - Resolve clauses until either the empty clause is produced or no resolvable clauses remain
 - Empty clause proves theorem (contradiction)

Short Example

- Given
 - Whoever can read is literate.
 - Dolphins are not literate.
 - Some dolphins are intelligent.
- Prove
 - Some who are intelligent cannot read.

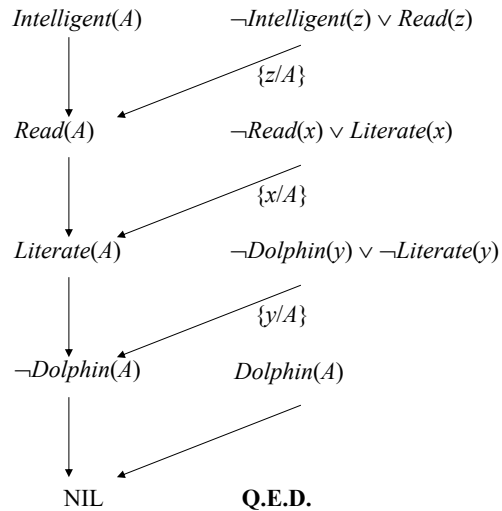
Convert to FOL

- Given
 - $\forall x [Read(x) \Rightarrow Literate(x)]$
 - $\forall x [Dolphin(x) \Rightarrow \neg Literate(x)]$
 - $\exists x [Dolphin(x) \wedge Intelligent(x)]$
- Prove
 - $\exists x [Intelligent(x) \wedge \neg Read(x)]$

Convert to Clause Form

- Axioms
 - $\neg Read(x) \vee Literate(x)$
 - $\neg Dolphin(y) \vee \neg Literate(y)$
 - $Dolphin(A)$ {A is a Skolem constant}
 - $Intelligent(A)$
- Negated Theorem
 - $\neg \exists x [Intelligent(x) \wedge \neg Read(x)]$, which yields in clause form
 - $\neg Intelligent(z) \vee Read(z)$

Resolution Proof Tree



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

57

Covering

- A rule “covers” a subset of examples in $\mathcal{D}$ if the left-hand side of the rule matches those examples.
- We learn a set of rules one rule at a time by finding rules that “cover” the data.

Algorithm 9.1 Sequential Covering

```

1: function SEQUENTIALCOVERING( $\mathcal{D}$ )
2:   partition  $\mathcal{D}$  into  $Pos$  and  $Neg$ 
3:   repeat
4:      $Rule \leftarrow \text{LearnOne}(Pos)$ 
5:      $LearnedRules \leftarrow LearnedRules \cup \{Rule\}$ 
6:      $Pos \leftarrow Pos - \{p \in Pos \mid \text{covered}(p, Rule)\}$ 
7:   until  $\text{Perf}(Rule, \mathcal{D}) > \theta$ 
8:    $LearnedRules \leftarrow \text{Sort}(LearnedRules, \text{Perf}([], \mathcal{D}))$ 
9:   return  $LearnedRules$ 
10: end function
  
```



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

58

One Rule

Algorithm 9.2 Learn One Rule

```

1: function LEARNONE( $\mathcal{D}$ )
2:    $bestHyp \leftarrow$  most general hypothesis
3:    $candHyps \leftarrow \{bestHyp\}$ 
4:   while  $candHyps \neq \emptyset$  do  $\triangleright$  Generate next more specific candidate hypotheses
5:      $allConstraints \leftarrow$  set of all  $(a = v)$   $\triangleright v$  is a value of  $a$  occurring in  $\mathcal{D}$ 
6:      $newCandHyps \leftarrow$ 
7:       for all  $h \in candHyps$  and  $c \in allConstraints$  do
8:         specialize  $h$  by adding  $c$ 
9:       end for
10:    remove duplicate, inconsistent, or non-max-specific  $h \in newCandHyps$ 
11:    // Update  $bestHyp$  only if performance improves
12:    for all  $h \in newCandHyps$  do
13:      if  $Perf(h, \mathcal{D}) > Perf(bestHyp, \mathcal{D})$  then
14:         $bestHyp \leftarrow h$ 
15:      end if
16:    end for
17:    // Update  $candHyps$  with the best from the candidates
18:     $candHyps \leftarrow$  up to the  $k$  best members of  $newCandHyps$ 
19:  end while
20:  return  $newRule$  of the form "IF  $bestHyp$  THEN  $prediction$ "
21:  //  $prediction$  is most frequent value of targets among matching examples
22: end function

```



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

59

FOIL

Prolog style:

$$P(x_1, x_2, \dots, x_k) \leftarrow$$

$$\ell_1, \dots, \ell_n$$

$$Q(v_1, \dots, v_r)$$

$$Equal(x_j, x_k)$$

Bindings

- Positive
- Negative

Algorithm 9.3 First Order Inductive Learning (FOIL)

```

1: function FOIL( $\mathcal{T}, \mathcal{P}, \mathcal{D}$ )
2:   //  $\mathcal{T}$  = target
3:   //  $\mathcal{P}$  = predicates
4:   //  $\mathcal{D}$  = examples
5:    $Pos \leftarrow \{x \in \mathcal{D} | x \text{ is a positive example}\}$ 
6:    $Neg \leftarrow \{x \in \mathcal{D} | x \text{ is a negative example}\}$ 
7:   while  $Pos \neq \emptyset$  do
8:      $newRule \leftarrow$  most general rule
9:      $newRuleNeg \leftarrow Neg$ 
10:    while  $newRuleNeg \neq \emptyset$  do
11:       $candLit \leftarrow GenCandidates(\mathcal{P})$ 
12:       $bestLit \leftarrow \arg \max_{\ell \in candLit} FOILGain(\ell, newRule)$ 
13:      add  $bestLit$  to  $newRule$  preconditions (i.e., antecedent)
14:       $newRuleNeg \leftarrow$  subset of  $newRuleNeg$  satisfying preconditions
15:    end while
16:     $learnedRules \leftarrow learnedRules \cup \{newRule\}$ 
17:     $Pos \leftarrow Pos - \{p \in Pos | covered(p, Rule)\}$ 
18:  end while
19:  return  $learnedRule$ 
20: end function

```



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

60

Selecting Predicates

- Similar to decision trees, select based on “information gain.”

$$FOILGain(\ell, Rule) = t \left(\lg \frac{p_1}{p_1 + n_1} - \lg \frac{p_0}{p_0 + n_0} \right)$$

- p_0 : # positive bindings in *Rule*
- n_0 : # negative bindings in *Rule*
- p_1 : # positive bindings in *Rule* + ℓ
- n_1 : # negative bindings in *Rule* + ℓ
- t : # prior positive bindings still covered by *Rule* + ℓ